

Project 1: Dynamic City or Farm Simulator

General Rules & Academic Integrity

- **Topics:** Practicing elements and concepts covered in class: Getting user input, data type conversion, manipulating lists/strings, implementing decision/control structures (if/else), loops, complex data types, functions, and file IO.
- **Concepts Used:** Only use concepts presented in the course to-date. Using advanced concepts or concepts from outside the course does not demonstrate understanding of the required material and may result in a zero for the assignment.
- **Restrictions:** Do not use **breaks**, global values (unless working with shallow copy), or modules other than random.
- **Resources:** You may reference the textbook, class materials, and tutoring (if you use a tutor, list their name in collaboration statement).
- **Academic Integrity: Do not use AI.** Asking AI, online searches, or people not associated with the course (results in a grade of zero) is not allowed and many times leads you to incorrect solutions. Specify any collaboration you had within the comment header. If you went to Boardman 138, state that.
- **Planning:** Start early. Planning out the program helps. Create a written plan of the processes in the program, how values will be passed, what values will be modified, processes, data flow, indicate all logic and inputs/outputs required.

II. Submission Requirements

- **Planning:** You will submit your planning document to brightspace prior to creating your program.
- **File Name:** yourName_Simulation.py (replace "yourName" with your actual name, e.g., lauraGurney_Simulation.py).
- **Comment Header:** Include the required comment header in your file and ensure data within the header is updated.
- **Folder Name:** Place the .py file into a folder named yourName_Simulation.
- **Submission:** Keep python files inside your assignment folder. Compress the folder (.zip) and submit the compressed file containing all project files.

Objective: Create a modular Python program that simulates the population dynamics of a chosen scenario (City or Farm) over a user-defined number of turns. The program must demonstrate proficiency in self-defined functions, complex data structures, structured file I/O (reading and writing different formats), and use of the random module.

Time Estimate: 2 weeks

Part 1: Initial Setup and File Structure

1. User Input and Scenario Choice

The program must begin by prompting the user for the following inputs:

- Scenario Choice: Enter either "City" or "Farm". This determines the initial data loaded and the terminology used throughout the simulation.
- Number of Turns: Enter the total number of turns the simulation should run.
 - Each row in the Weather file represents a "turn" to calculate the simulation upon.

2. File Requirements and I/O

The program must interact with three external files:

File Name	Format	Purpose
start_data.txt	Text	Load initial population counts (People, Pets/Livestock, Pests).
weather.csv	CSV	Provides sequential weather data for each turn.
simulation_log.txt	Text	Logs turn-by-turn events AND the final summary report.
state.txt	Text	Saves the simulation state for later continuation

3. Data Structure Design

The data should be represented by a master dictionary containing the simulation state. The data will contain the following. Create a dictionary formatted to hold the following example data.

Example Data	Notes
"City" or "Farm"	Used for display/logging.
Integer	Current turn number.
{'count': 10000, 'name': 'People'}	Tracks People.
{'count': 5000, 'name': 'Pets'} or {'count': 200, 'name': 'Livestock'}	Tracks Pets/Livestock.
{'count': 1500, 'name': 'Pests'}	Tracks Pests.
Comma separated data	All data read from weather.csv

4. Initialization File Parsing

- Implement functions to read and parse external files:
 - The start_data.txt file based on the user's scenario choice as received by input. The data file will have both the farm and the city data, but must load data from the file based on the choice.
 - How many turns, as received by input up to the length of the weather file's records. Each row in the weather file is a "turn".
 - Open an external file to record data to at the end of the simulation program.

Example data: formatting will remain, data may change.

[CITY],people:10000,pets:5000,pests:1500

[FARM],people:50,livestock:200,pests:50

- The weather.csv file that will have the weather for each week of the year.
 - Purpose: Load sequential weather data for the duration of the simulation.
 - Read Logic: The student must read this file into a list of dictionaries or list of tuples within the main state dictionary for later, indexed access during the simulation loop.
 - Each row in the weather file is a turn in the game.

Example data: formatting will remain, data may change.

```
Wk,RainFall,Heat
01,1.10,56
02,.53,36
03,1.3,60
...
```

- State and Log Files (Read/Write/Append)
 - State File: The state of the game dictionary is saved and loaded (read/write) to allow the user to continue the simulation later.
 - History Log: An append-only text file recording the outcome of each turn.

4. Simulation Logic

The most complex function is the one controlling the turns and takes the state input. This will execute a series of conditional checks based on external data.

A. Apply Weather Rules

- Get the user to indicate how many turns they want to have the simulation run and limit it to the length of rows in the weather file. Create the logic so that if a different weather file is used, then it will work with the number of rows in that file. (e.g. weather.csv right now is 52 rows of weather, what if you get a file with 150 rows?)
- Pass data to a function (e.g., weather_impact) that reads the current turn's weather data from the loaded CSV list and applies the following rules:
 - Low Rainfall: If Rainfall_cm is less than a predefined threshold (e.g., 1.0 cm):
 - Pets/Livestock decrease by 10%.
 - High Rain and High Heat: If Rainfall_cm is greater than a high threshold (e.g., 4.0 cm) AND Heat_C is greater than a high threshold (e.g., 30 degrees C):
 - Pests increase by a random amount (e.g., 10 percentage to 20 percentage) and trigger the pest function
 - Pest Overpopulation function and rules
 - A function must implement the cascading effect of an uncontrolled pest population:
 - Pest Threat Threshold: Calculate if Pests are 30 percentage greater than the combined count of People

and Pets/Livestock. If Pest Count is greater than $1.3 \times (\text{People} + \text{Pets/Livestock})$, then **both** People and Pets/Livestock populations decrease by 5%

- Standard Simulation and Logging
 - Apply a user entered standard, base-level change for a small, random birth and death rate for all groups
 - Log the turn results (population counts and the event outcome) to the history log file.

Data to Record	Beginning Count (From start_data.txt)	Ending Count (Final Turn data)
Scenario	Scenario Choice	Scenario Choice
People	Initial People Count	Final People Count
Pets/Livestock	Initial Population Count	Final Population Count
Pests	Initial Pest Count	Final Pest Count
Turns Run		Total Turns Run